

Звіт

до лабораторної роботи №7

Об'єктно-орієнтоване програмування. Java

Інтерфейси та множинна реалізація інтерфейсів у Java

Студент: Ісаєв Захар Ігорович

Група: КН-924В

Викладач: Володимир Миколайович Савченко

Харків - 2026

Мета роботи

Сформуванати практичні навички проєктування семантично коректних моделей Java з використанням інтерфейсів, застосування інтерфейсів як контрактів і типів, реалізації одного або кількох інтерфейсів у конкретних класах, а також використання множинної реалізації інтерфейсів для подання незалежних ролей або можливостей.

Опис предметної галузі

У даній лабораторній роботі розглядається предметна область інформаційних сервісів кампусу. Для моделювання було обрано сервіси, які допомагають студенту отримувати важливу інформацію та планувати навчальні події. До таких сервісів належать нагадування про іспит, бронювання аудиторії та консультація викладача.

На відміну від лабораторної роботи №6, де була побудована модель оцінювання студентських робіт через абстрактний клас `Assessment`, у лабораторній роботі №7 головний акцент зроблено не на спадкуванні, а на інтерфейсах. Це пов'язано з тим, що `ExamReminder`, `RoomBooking` і `ConsultationSlot` не є різновидами одного конкретного об'єкта, але вони можуть мати спільні можливості.

У роботі використано два інтерфейси: `Notifiable` і `Schedulable`. Інтерфейс `Notifiable` описує можливість повідомляти студента, а інтерфейс `Schedulable` описує можливість запланувати подію на певний час. Клас `ExamReminder` реалізує можливість повідомлення, `RoomBooking` реалізує можливість планування, а `ConsultationSlot` реалізує обидві можливості одночасно.

Чому вибрані інтерфейси є семантично обґрунтованими

Інтерфейс `Notifiable` описує об'єкти, які можуть повідомляти студента про певну подію. Він є доречним для цієї предметної області, тому що інформаційні сервіси кампусу часто повинні передавати студенту важливі повідомлення.

Наприклад, `ExamReminder` повідомляє студента про дату іспиту, а `ConsultationSlot` може повідомити студента про консультацію з викладачем.

Інтерфейс `Schedulable` описує об'єкти, які можна запланувати на певну дату та час. Він також підходить для цієї моделі, тому що в кампусі потрібно планувати аудиторії, консультації та інші навчальні події.

Таким чином, обидва інтерфейси описують саме поведінкові можливості об'єктів, а не дублюють один конкретний клас.

Які класи реалізують кожен інтерфейс

Інтерфейс `Notifiable` реалізують два класи: `ExamReminder` і `ConsultationSlot`.

Клас `ExamReminder` повідомляє студента про іспит. Клас `ConsultationSlot` повідомляє студента про консультацію.

Інтерфейс `Schedulable` реалізують два класи: `RoomBooking` і `ConsultationSlot`.

Клас `RoomBooking` дозволяє запланувати бронювання аудиторії. Клас `ConsultationSlot` дозволяє запланувати консультацію викладача.

Який клас реалізує кілька інтерфейсів і чому це є змістовним

Клас `ConsultationSlot` реалізує два інтерфейси: `Notifiable` і `Schedulable`.

Це є змістовним, тому що консультація має дві різні можливості.

По-перше, консультацію можна запланувати на певний час. По-друге, про консультацію можна повідомити студента.

Тобто `ConsultationSlot` можна розглядати як об'єкт, який планується, і як об'єкт, який повідомляє студента. Саме цей клас демонструє множинну реалізацію інтерфейсів у Java.

Де саме в клієнтському коді використовуються типи інтерфейсів

Типи інтерфейсів використовуються у класі `Main`.

Для демонстрації повідомлень у програмі використовується масив типу `Notifiable[]`. У цьому масиві зберігаються об'єкти `examReminder` і `consultationSlot`. Обидва ці об'єкти можна обробляти однаково, тому що вони реалізують інтерфейс `Notifiable`.

Для демонстрації планування використовується масив типу `Schedulable[]`. У цьому масиві зберігаються об'єкти `roomBooking` і `consultationSlot`. Вони також можуть оброблятися однаково, тому що реалізують інтерфейс `Schedulable`.

Також у `Main` показано, що один і той самий об'єкт `consultationSlot` можна використати через два різні інтерфейсні посилання: як `Notifiable` і як `Schedulable`.

Це показує, що клієнтський код працює не тільки з конкретними класами, а й через типи інтерфейсів.

Чому вибране розв'язання краще виражається через інтерфейси, ніж лише через ієрархію класів

Класи `ExamReminder`, `RoomBooking` і `ConsultationSlot` мають різне призначення.

`ExamReminder` відповідає за нагадування про іспит. `RoomBooking` відповідає за бронювання аудиторії. `ConsultationSlot` відповідає за консультацію викладача.

Ці класи не є різновидами одного спільного базового класу, тому створювати для них штучну ієрархію успадкування було б не дуже правильно. Але вони можуть мати спільні можливості.

Наприклад, деякі об'єкти можуть повідомляти студента, а деякі можуть плануватися на певний час. Тому інтерфейси краще підходять для цієї моделі, бо вони описують не те, чим є об'єкт, а те, що він може робити.

Короткий висновок щодо додаткового завдання

Для додаткового завдання взяв модель з лабораторної роботи №6.

У тій роботі були класи `Assessment`, `Exam`, `QuizAssessment` і `CourseProject`. Класи `Exam`, `QuizAssessment` і `CourseProject` наслідуються від абстрактного класу `Assessment`. Це логічно, тому що вони мають спільні поля: назву курсу, ім'я студента, групу, максимальний бал і отриманий бал.

Але частину поведінки цієї моделі можна описати через інтерфейси.

Можна запропонувати інтерфейс `Evaluable`, який описує можливість оцінювання результату. Для нього можна використати метод `evaluateResult()`.

Також можна запропонувати інтерфейс `Scorable`, який описує роботу з балами. Для нього можна використати методи `getPercentage()` і `hasMinimumPercentage()`.

Таке перепроєктування частково покращує модель. Абстрактний клас `Assessment` все ще потрібний, бо він зберігає спільні поля та загальні методи. Але інтерфейси дозволяють краще показати спільні можливості об'єктів.

Таблиця використаних механізмів Java

Механізм	Приклад використання
<code>interface</code>	створення інтерфейсів <code>Notifiable</code> і <code>Schedulable</code>
<code>implements</code>	реалізація інтерфейсів у класах <code>ExamReminder</code> , <code>RoomBooking</code> , <code>ConsultationSlot</code>
<code>@Override</code>	перевизначення методів <code>notifyStudent()</code> і <code>schedule()</code>
інтерфейс як тип	використання <code>Notifiable[]</code> і <code>Schedulable[]</code> у класі <code>Main</code>
множинна реалізація інтерфейсів	клас <code>ConsultationSlot</code> реалізує два інтерфейси
поліморфізм	робота з різними об'єктами через один тип інтерфейсу
інкапсуляція	поля класів оголошені як <code>private</code>

UML та робота програми

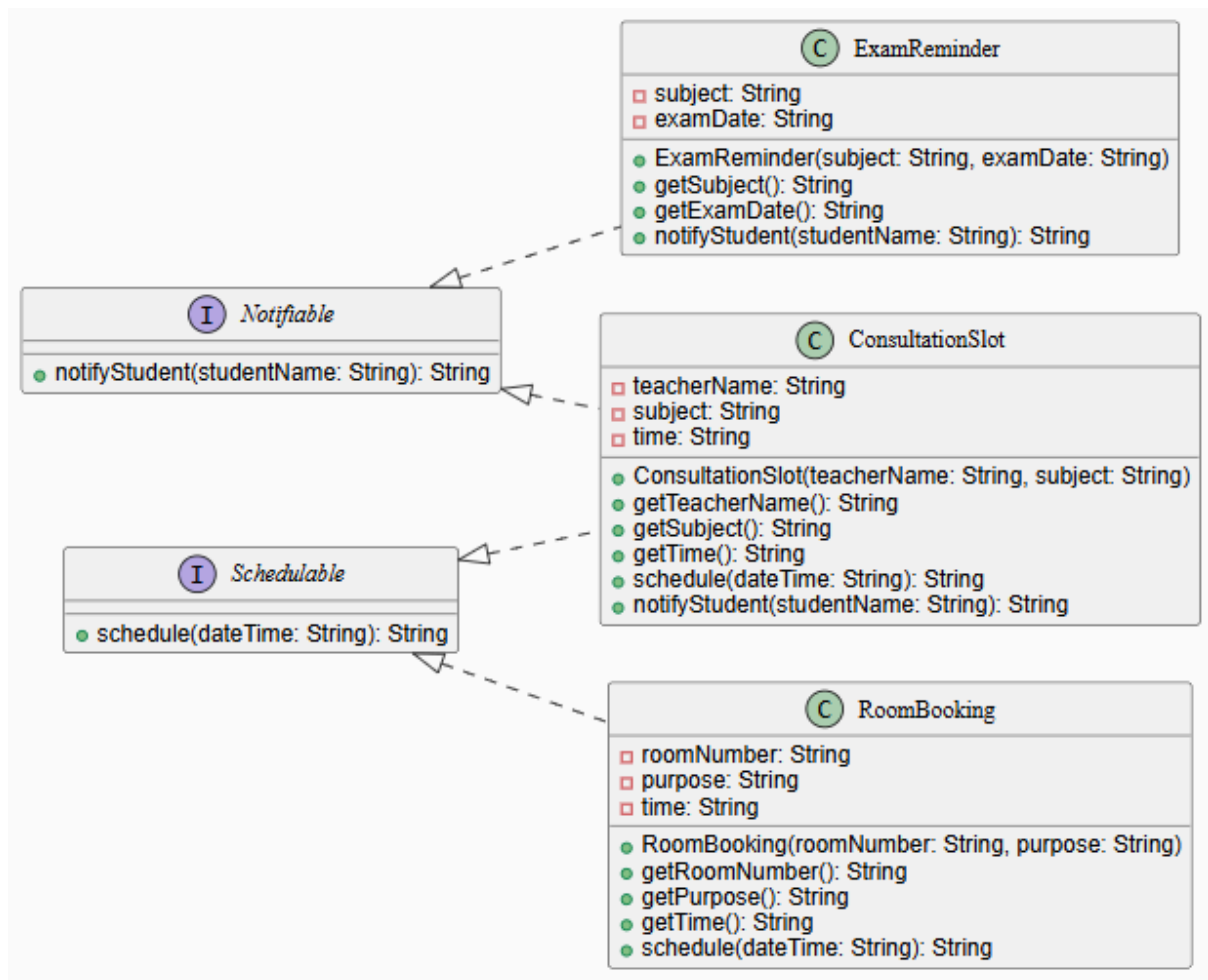


Рис. 1 - UML інформаційних сервісів кампусу

```

Lab 07
Student: Isaiev Zakhar Ihorovych
Variant: 3
Topic: Campus information services

Notifications:
Student Zakhar, exam in Object-Oriented Programming will be on 25.05.2026.
Student Zakhar, consultation in Java interfaces is not scheduled yet.

Scheduling:
Room 924 is scheduled for Java laboratory work at 20.05.2026 12:30.
Consultation with V. Volodymirivna in Java interfaces is scheduled at 20.05.2026 12:30.

One object through two interfaces:
Consultation with V. Volodymirivna in Java interfaces is scheduled at 21.05.2026 14:00.
Student Zakhar, consultation in Java interfaces with V. Volodymirivna will be at 21.05.2026 14:00.
  
```

Рис. 2 - Робота програми

Висновок

У ході лабораторної роботи №7 було розроблено модель інформаційних сервісів кампусу з використанням інтерфейсів у Java. Було створено два інтерфейси: `Notifiable` для повідомлення студента та `Schedulable` для планування подій. Також було реалізовано три класи: `ExamReminder`, `RoomBooking` і `ConsultationSlot`. Клас `ConsultationSlot` демонструє множинну реалізацію інтерфейсів, оскільки він може одночасно планувати консультацію та повідомляти студента про неї. У класі `Main` було показано роботу з об'єктами через типи інтерфейсів, що робить програму більш гнучкою.